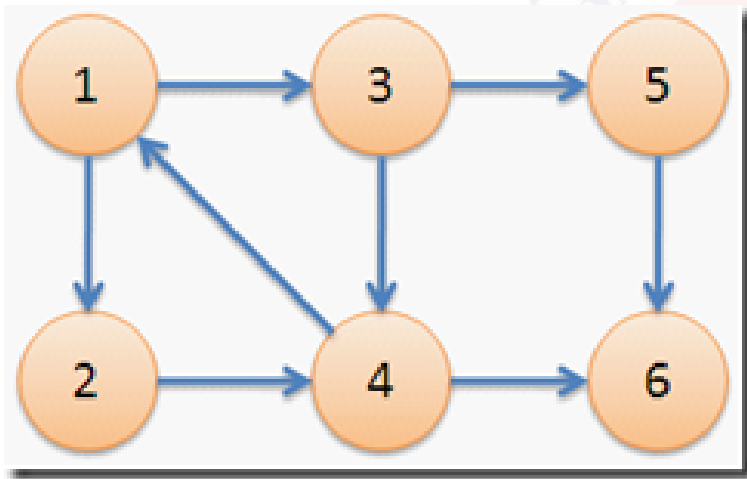


强连通及缩点 Tarjan算法及应用

主讲：黄凤林

基本概念

- ✧ 在有向图G中，
- ✧ 两个顶点间至少存在一条路径能**相互到达**，称**两个顶点强连通**
- ✧ 有向图G的**每两个顶点都强连通**，称G是一个强连通图。
- ✧ 非强连通图有向图的**极大强连通子图**，称为**强连通分量**



在右图中，子图 $\{1,2,3,4\}$ 为一个强连通分量，因为顶点 1,2,3,4 两两可达。 $\{5\},\{6\}$ 也分别是两个强连通分量。

强连通分量的算法--tarjan

✧ **算法思想：** dfs遍历G中的每个顶点，用：

dfn[i]表示dfs时**达到顶点i**的时间（**时间戳**）；

low[i]表示i所能直接或间接**返回到最小的顶点**时间。（实际操作中low[i]的值不一定最小，但不影响程序的最终结果）

✧ **DFS流程**

(1)程序开始时，t(时间)初始化为0，在dfs遍历到v时， $low[v]=dfn[v]=++t$ ，v入栈(这里的栈为自定义栈，而不是dfs的递归时系统栈)；

(2)遍历v所能直接达到的顶点k。如果k没被访问，那么先dfs遍历k，遍历完后取 $low\{v,k\}$ 最小值($low[v]=\min(low[v],low[k])$)；如果k在栈里，那么直接 $low[v]=\min(low[v],dfn[k])$ 或 $low[v]=\min(low[v],low[k])$ 均可。(就是这里会使得low[v]不一定最小但不会影响到结果；因为求完后的low[v]一定小于dfn[v])。

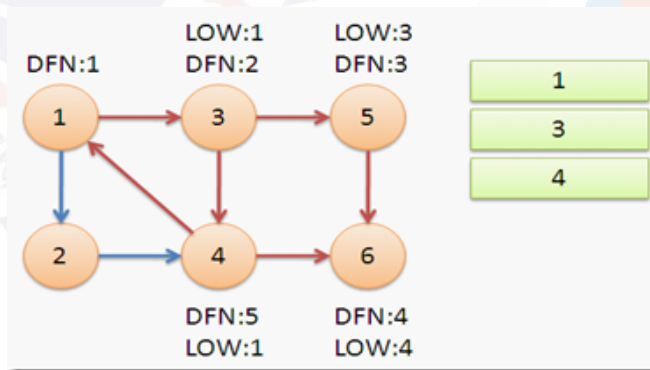
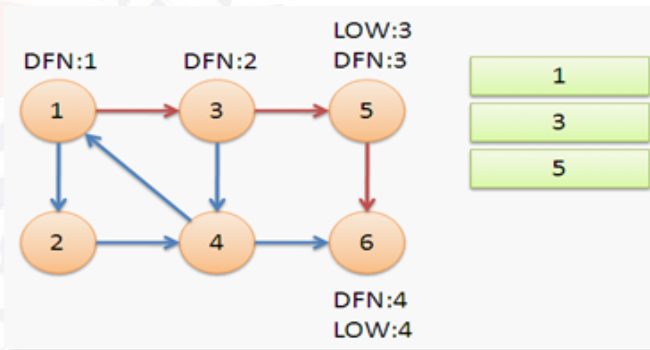
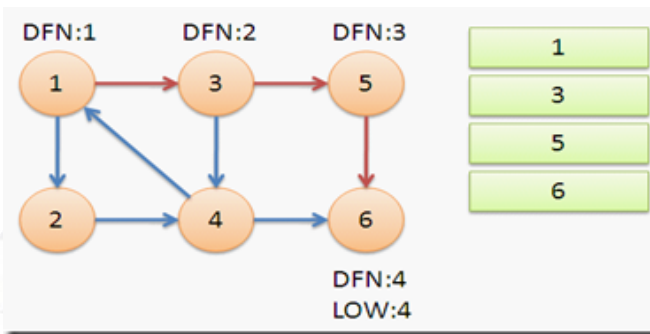
(3)遍历完所有的k以后，如果 $low[v]==dfn[v]$ 时(到达时间=返回时间)，则栈里v以及v以上到栈顶的顶点全部出栈，且刚刚出栈的就是一个极大强连通分量。

算法流程演示

1、从节点1开始DFS，把遍历到的节点加入栈中。搜索到节点 $u=6$ 时，由于 $dfn[6]=low[6]=4$ ，找到了一个强连通分量。退栈到 $u=v$ 为止， $\{6\}$ 为一个强连通分量。

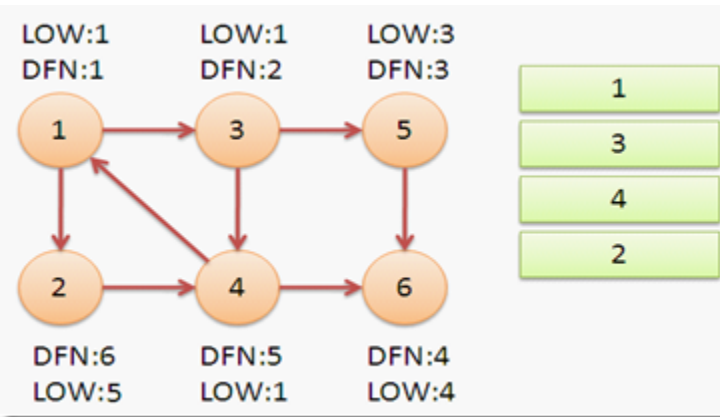
2、回溯返回节点5，发现 $dfn[5]=low[5]=3$ ，找到了一个强连通分量，退栈后 $\{5\}$ 为一个强连通分量。

3、返回节点3，继续搜索到节点4，把4加入堆栈。发现节点4向节点1有后向边，节点1还在栈中，所以 $low[4]=1$ 。节点6已经出栈， $(4,6)$ 是横叉边，不管他；返回3， $(3,4)$ 为树枝边，所以 $low[3]=low[4]=1$ 。



算法流程演示

4、继续回到节点1，最后访问节点2。访问边(2,4)，4还在栈中，所以 $low[2]=dfn[4]=5$ 。返回1后，发现 $dfn[1]=low[1]$ ，把栈中节点全部取出，组成一个连通分量{1,3,4,2}。



至此，算法结束。经过该算法，求出了图中全部的三个强连通分量{1,3,4,2},{5},{6}。

可以发现，运行Tarjan算法的过程中，每个顶点都被访问了一次，且只进出了一次堆栈，每条边也只被访问了一次，所以该算法的时间复杂度为 $O(N+M)$ 。

→ Tarjan核心代码

```
void tarjan(int u){
    dfn[u]=low[u]=++t; // 为节点u设定次序编号和Low初值
    stack[++top]=u; // 将节点u压入栈中
    instack[u]=true;
    for(int i=head[u];i;i=edge[i].next){ // 枚举每一条边
        if(!dfn[edge[i].to]){ // 如果节点未被访问过
            tarjan(edge[i].to); // 继续向下找
            low[u]=min(low[u],low[edge[i].to]);
        } else if(instack[edge[i].to]) // 如果节点在栈内
            low[u]=min(low[u],dfn[edge[i].to]);
    }
    if(dfn[u]==low[u]){ // 如果节点u是强连通分量的根
        ans++; // 强连通数量加1
        while(stack[top]!=u){
            printf("%d ",stack[top]); instack[stack[top--]]=false;
        }
        printf("%d\n",stack[top]); instack[stack[top--]]=false;
    }
}
```

参考博客: <http://www.cppblog.com/sosi/archive/2010/09/26/127797.aspx>

→ Tarjan的应用1—缩点

- ✧ 缩点：就是把有向有环图通过强连通分量缩成有向无环图
- ✧ 针对有向无环图可以做很多事情，比如做图上dp或最短路
- ✧ Tarjan缩点写法

```
if(dfn[u]==low[u]){ //如果节点u是强连通分量的根
    ans++; //强连通数量加1
    int temp;
    do{
        temp=stack[top--];
        instack[temp]=false;
        newNode[temp]=ans;
        //newNode[i]=j表示原图i点在新缩完点后的有向无环图中的节点编号为j
    }while(temp!=u)
```




洛谷P3387

✧ 题目描述

给定一个 n 个点 m 条边有向图，每个点有一个权值，求一条路径，使路径经过的点权值之和最大。

你只需要求出这个权值和。

注：**允许多次经过一条边或者一个点**，但是，重复经过的点，权值只计算一次。

✧ 数据范围：

对于 100% 的数据， $n \leq 104$ ， $m \leq 105$ ，点权 $\in [0, 1000]$

➔ 问题分析

- ✧ 这是一个经典的**有向图先缩点，再求最大值**问题
- ✧ 我们可以用tarjan对原图就行缩点，变成一个有向无环图
- ✧ 然后我们再对有向无环图按照**拓扑序**进行dp（dfs序）
- ✧ 用dp[i]表示**到达i点的最大路径值**，则
- ✧ $dp[i] = \max\{dp[k]\}$, k表示k到i有边，最后 $ans = \max\{dp[i]\}$



→ 参考代码

```
void tp(){//拓扑排序求dp[i]
    queue<int> q;
    int ans=0;
    for(int i=1;i<=cnt;i++){
        if(rd[i]==0){
            q.push(i);dp[i]=newA[i];
            ans=max(ans,dp[i]);
        }
    }
    while(!q.empty()){
        int u=q.front();q.pop();
        for(int i=head2[u];i;i=ed[i].next){
            int v=ed[i].to;
            if(dp[v]<dp[u]+newA[v]){
                dp[v]=dp[u]+newA[v];
                ans=max(ans,dp[v]);
            }
            rd[v]--; if(rd[v]==0) q.push(v);
        }
    }
    printf("%d\n",ans);
}
```



→ Tarjan练习题

- ✧ 洛谷P3387[缩点]缩点
- ✧ 洛谷P2341
- ✧ 洛谷P2863
- ✧ Bzoj1179
- ✧ 洛谷P3627
- ✧ 洛谷P2661 信息传递
- ✧ 洛谷P3119
- ✧ 洛谷P2921

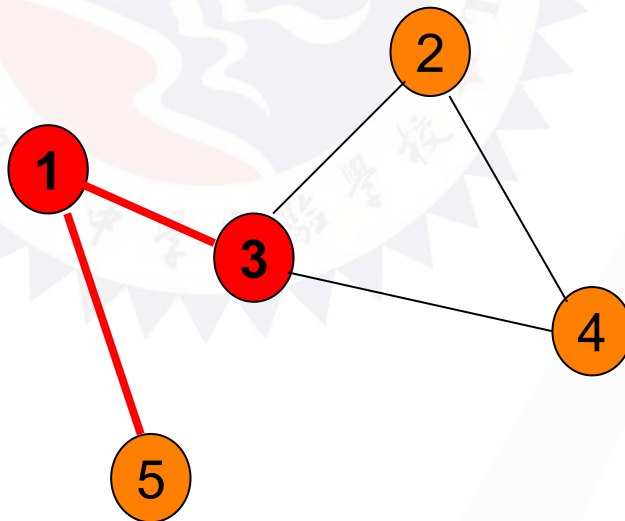
→ Tarjan应用2—割点割边

无向图的割点和割边(桥)

一.基本概念

1.割边：是存在于无向图中的这样的一条边，如果去掉这一条边，那么**整张无向图会分为两部分**，这样的一条边称为桥。无向连通图中，如果删除某边后，**图变成不连通**，则称该边为桥。

2.割点：无向连通图中，如果**删除某点后，图变成不连通**，则称该点为割点。



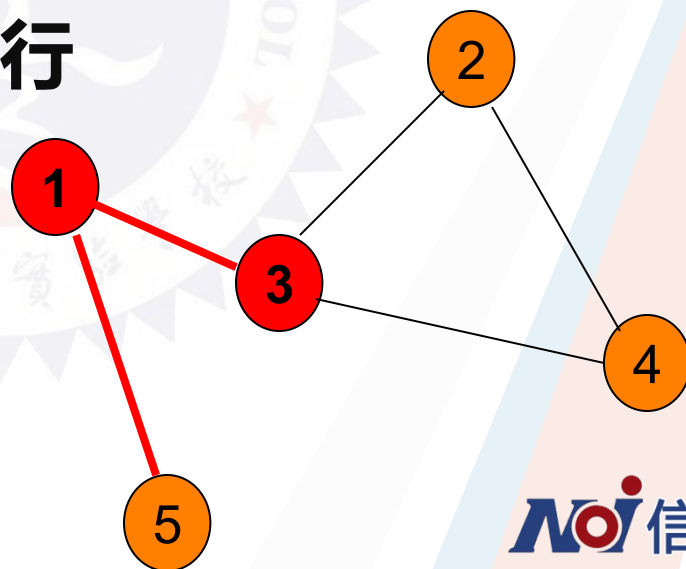
➔ 割点求法

✧ 割点：

(1) 当前节点U为树根的时候，条件是“要有多余一棵子树”。
(如果只有一颗子树，去掉这个点也没有影响，如果有两颗子树，去掉这点，两颗子树就不连通了)

(2) 当前节点U不是树根的时候，条件是“ $\text{low}[v] \geq \text{dfn}[u]$ ”
也就是在u之后遍历的点，能够向上翻，最多到u，如果能翻到u的上方，那就有环了，去掉u之后，图仍然连通。

✧ 保证v向上最多翻到u才行

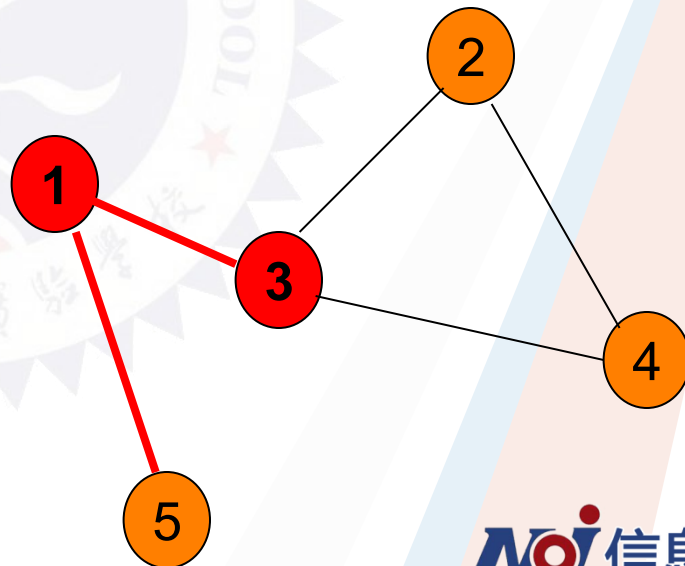


桥的求法

桥—割边

若是一条无向边 (u, v) 是桥，当且仅当无向边 (u, v) 是树枝边的时候，需要满足 $dfn(u) < low(v)$ 。

也就是 v 向上翻不到 u 及其以上的点，那么 $u-v$ 之间一定能够有1条或者多条边不能删去，因为他们之间有一部分无环，是桥。如果 v 能上翻到 u 那么 $u-v$ 就是一个环，删除其中一条路径后，任然是连通的。



➔ 求桥的注意事项

(1)求桥的时候：因为边是无方向的，所以父亲孩子节点的关系需要自己规定一下。

在tarjan的过程中if (v不是u的父节点)
 $\text{low}[u] = \min(\text{low}[u], \text{dfn}[v]);$

因为如果v是u的父亲，那么这条无向边就被误认为是环了。

(2)找桥的时候：**注意看看有没有重边**，有重边的边一定不是桥，也要避免误判。

➔ 割点割边核心代码

```

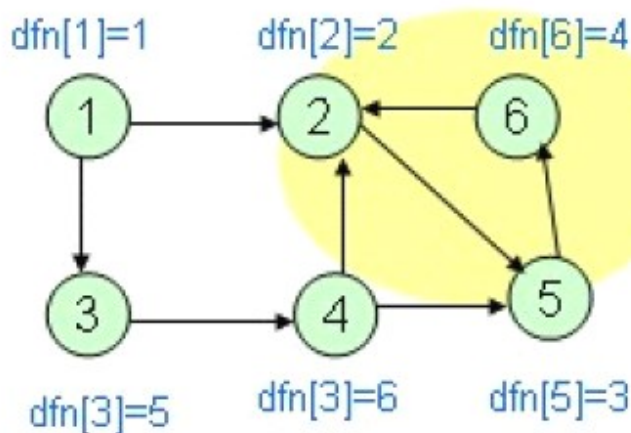
bool isap[maxn];
struct node{
    int u,v;
} cut_edge[maxn];
void tarjan(int u,int fa){
    low[u]=dfn[u]=++t;
    for(int i=head[u];i;i=edge[i].next){
        int v=edge[i].to;
        if(v==fa) continue;
        if(!dfn[v]){
            tarjan(v,u);
            low[u]=min(low[u],low[v]);
            if(du[v]>1 && dfn[u]<=low[v]) isap[u]=1;
            if(dfn[u]<low[v]){cut_edge[++k].u=u;cut_edge[k].v=v;}
        }
        else low[u]=min(low[u],dfn[v]);
    }
}

```

为什么不用判断点v是否在栈中呢?

是否入栈解释

根据dfn可以看出搜索的顺序是1→2→5→6形成一个强连通分量(2, 5, 6)，于是开始退栈，回溯到1从3出发到达4，此时如果直接用dfn[2]更新low[4]的话，会得到low[4]=2，变小后而与dfn[4]不再相等，不能退栈，这与最后的4形成一个单独强连通分量是不符合的，所以，不在栈中的点，不能用来更新当前点的low[]值，为什么无向图不用标记呢，那时因为，边是无向的，有边从4→2同时也必有边2→4 由于2 之前被标记过，而遍历到当前结点4 又不是通过w(2, 4)这条边过来的，则必还存在另一条路径可以使2 和4 是相通的，(即图中的4-3-1-2)，从而2, 4 是双连通的。





➔ 最近公共祖先

- ✧ 什么是最近公共祖先？
- ✧ 回顾RMQ转最近公共祖先的求法：
- ✧ tarjan也有一个求最近公共祖先的算法。这个tarjan与强连通分量的tarjan有很大区别，仅仅因为这两个算法都是tarjan这个人创造的而已，求公共祖先的tarjan算法是一个离线算法，用并查集实现的。
- ✧ 练习题目：校门外的树



→ Tarjan求最近公共祖先

```
void tarjan_lca(int x){
    father[x]=x; // 构建以x为根的集合
    visit[x]=true;
    for(int i=head[x]; i>0; i=e[i].next){ // 枚举相邻边, 递归子树
        if(!visit[e[i].to]){ // 如果当前节点没有访问过
            tarjan_lca(e[i].to); // 递归子树
            father[e[i].to]=x; // 回溯集合合并子树
        }
    }
    for(int i=head_q[x]; i>0; i=e_q[i].next){ // 处理包含x点对询问且以遍历最近公共祖先
        if(visit[e_q[i].to]){ // 如何当前询问对的节点遍历过
            int lca=find(e_q[i].to);
            printf("%d-->%d: %d\n", x, e_q[i].to, lca);
        }
    }
}
```

→ 最近公共祖先算法比较

- ✧ 最近公共祖先算法：RMQ、Tarjan、树上倍增
- ✧ RMQ和Tarjan优点：算法思想简单，容易快速求出 $u-v$ 的最近公共祖先节点
- ✧ 缺点：不能够求解 $u-v$ 路径上权值**最小值或最大值**问题。如2013 noip day1-3。
- ✧ 如何才能快速求出LCA且路径权值的最值问题呢？

树上倍增

The background features a central point from which several wide, colorful rays (in shades of blue, orange, red, and grey) radiate outwards. Faint, light green binary code (0s and 1s) is scattered across the background, particularly concentrated around the central text.

Thank you!